A

**PROJECT REPORT**

On

# Design and Implementation of a Hostel Mess Management System using MySQL

Submitted in partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**

in

**Computer Science & Engineering**


**Submitted By:**

AYUSH SRIVASTAVA

Registration Number: 2502052273

Section: P3


**Subject Name: Relational Database Management Systems**




MANIPAL UNIVERSITY JAIPUR


**School of Computer Science and Engineering**

**Department of Computer Science and Engineering**

**Manipal University Jaipur**

Jaipur, Rajasthan


July–December 2026

# Table of Contents


Table of Contents 2
**1. Introduction** 3
**2. Objectives** 4
**3. Features and Functionality** 5
3.1 Student Module 5
3.2 Attendance and Automatic Closure of Meals 5
3.3 Meal Rating Module 5
3.4 Management Module 5
3.5 Database Operations Employed 6
**4. Tools and Technologies Used** 7
**5. ER Diagram and Relational Model** 8
5.1 Entities and Attributes 8
5.2 Relationships 9
5.3 Relational Schema 11
5.4 Foreign Keys 11
5.5 Normalisation 12
**6. Database Implementation** 13
6.1 Table Creation, Data Types and Constraints 13
6.2 Population of the Database 14
6.3 Data Manipulation Performed by the Application 14
**7. SQL Queries and Outputs** 16
7.1 Data Definition and Database Structure 16
7.2 Data Manipulation: Insertion, Updation and Deletion 18
7.3 Selection and Joins 20
7.4 Grouping, Aggregation and Subqueries 21
7.5 Functions 23
7.6 Views 23
**8. Advanced Database Operations** 26
8.1 Transactions 26
8.2 Triggers 27
8.3 Stored Procedures and Function 29
8.4 User Privileges 32
8.5 Scheduled Events 33
**9. Screenshots** 34
**10. Conclusion** 39
**11. References** 40

# 1. Introduction

Manipal University Jaipur operates two hostel messes, namely BluDove and BluSpring. Under the existing arrangement, every resident student pays a common annual mess fee and is required to register with one of the two messes for a fixed period of two months. During this period the student may take meals only at the registered mess. This arrangement restricts the choice available to students and, since neither mess has advance information about the number of students who will actually attend a given meal, the quantity of food prepared is estimated rather than planned. A considerable portion of the prepared food is consequently wasted.

The present project, titled **Design and Implementation of a Hostel Mess Management System using MySQL**, addresses these limitations through a relational database application. The annual fee remains unchanged; however, for each of the four daily meals (breakfast, lunch, hi-tea and dinner) a student may select either mess, or indicate that the meal will be skipped, up to seven days in advance. Since every selection is recorded in the database, each mess is able to determine precisely how many meals are to be prepared. The quantity prepared is computed as the number of confirmed bookings together with a buffer of thirty percent for students who arrive without a booking. Attendance at each meal is recorded by the face-recognition terminals already installed at both messes, students rate the meals they have eaten, and the surplus food remaining after each meal is allocated automatically to non-governmental organisations (NGOs) operating near the campus.

The core of the system is a MySQL 8 database consisting of fifteen tables in the Third Normal Form, fifteen triggers, seven stored procedures, one stored function, eight views and one scheduled event. All significant business rules, such as the restriction of one selection per student per meal, the prohibition of bookings during a declared leave period and the requirement that a student be allotted a hostel corresponding to the student's gender, are enforced within the database itself. A web-based interface for students and an analytical dashboard for the management of both messes have been developed over the database by means of a Python (FastAPI) service; nevertheless, the logic of the application resides in SQL and can be demonstrated entirely through MySQL Workbench.

## 2. Objectives

The principal objectives of the project are as follows:

1. To design a normalised relational database that records students, hostels, academic programmes, messes, meals, bookings, attendance and donations without redundancy.
2. To permit every student to select a mess independently for each meal of the day, in place of a fixed two-month registration.
3. To provide each mess with an accurate forecast of the quantity of food to be prepared, derived from bookings through a database view.
4. To enforce data integrity through primary keys, foreign keys, unique keys, CHECK constraints and triggers, so that invalid data cannot be stored by any application.
5. To implement multi-step operations, such as suspending a student's meals during leave, as stored procedures executed within transactions, thereby guaranteeing atomicity.
6. To close every meal automatically at the end of its serving period by means of a scheduled database event, and to allocate the surplus food equitably among partner NGOs.
7. To collect students' ratings of the meals they have eaten, admitting a rating only for a meal actually served, so that each mess can monitor the quality of its food.
8. To furnish the management of each mess with information on revenue, food cost, profit, wastage and absenteeism.
9. To secure the database by means of parameterised queries and a database user granted only the minimum necessary privileges.

**Expected outcome:** a fully functional, normalised database integrated with a web application, in which every transaction performed by a user is reflected immediately in the database and every business rule is demonstrably enforced by the database management system.

# 3. Features and Functionality

## 3.1 Student Module

- **Authentication:** the student logs in with a mobile number and a six-digit one-time password. A mobile number not yet present in the database is registered automatically at the first login, which in turn invokes the enrolment trigger.
- **Menu display:** the menu of both messes is displayed for each meal of the day.
- **Meal booking:** for each meal the student selects BluDove, BluSpring or Skip, for the current day and the following seven days. A change of selection updates the existing record by means of `INSERT … ON DUPLICATE KEY UPDATE`, since a unique key permits only one record per student per meal.
- **Leave management:** the student may suspend meals for a period of leave through the stored procedure `sp_pause_mess` and may resume them early through `sp_resume_mess`.

## 3.2 Attendance and Automatic Closure of Meals

- **Recording of attendance:** both messes have face-recognition terminals at the serving counter. On recognising a student, the terminal inserts a record into the `attendance` table; a student served without a booking at that mess is identified as a walk-in.
- **Automatic closure of a meal:** the scheduled event `ev_close_meals` runs every ten minutes and invokes `sp_close_finished_meals`, which closes every meal whose serving period has ended by calling `sp_close_meal`. The latter computes the surplus and records its donation to an NGO. No manual action is required.

## 3.3 Meal Rating Module

- After a meal recorded by the face-recognition terminal, the student may rate it from one to five stars on the booking page.
- The rating is stored in the table `ratings`. A BEFORE trigger admits the rating only if a corresponding attendance record exists, and a CHECK constraint restricts the value to the range one to five.
- The view `v_mess_ratings` provides the average rating of each mess for each meal, which the management dashboard presents as a ratings board.

## 3.4 Management Module

- Revenue, food cost, profit, profit margin, wastage and absenteeism of each mess over the preceding seven days, obtained from the view `v_mess_earnings`.

- Bookings received for the following day and the corresponding quantity to be prepared, obtained from the view `v_daily_forecast`, for the planning of procurement.
- The average rating of each mess for each meal over the preceding seven days, obtained from the view `v_mess_ratings`.
- A meal-wise summary of the current day, the distribution of hostels between the two messes, the record of donations, and an activity log maintained exclusively by triggers.

## 3.5 Database Operations Employed

| Category of operation | Application in the project |
|---|---|
| DDL: CREATE TABLE, VIEW, TRIGGER, PROCEDURE, FUNCTION; CREATE USER; GRANT | Definition of the schema (db/schema.sql) and of database users |
| DDL: ALTER TABLE, DROP; DML: INSERT, UPDATE, DELETE, INSERT … ON DUPLICATE KEY UPDATE | Login, booking, attendance, ratings, leave management |
| Joins, subqueries, correlated subqueries, UNION, GROUP BY, HAVING, WITH ROLLUP | Views and reports |
| Transactions with rollback on error and savepoints; cursors; loops | sp_pause_mess, sp_resume_mess, sp_close_meal, sp_close_finished_meals, sp_demo_week |
| Scheduled events (CREATE EVENT) | ev_close_meals: automatic closure of finished meals |
| CREATE USER, GRANT, REVOKE | Least-privilege application user; read-only management user |
| BEFORE and AFTER triggers; SIGNAL SQLSTATE | Enforcement of business rules and maintenance of the audit log |

# 4. Tools and Technologies Used

| Tool / Technology | Purpose |
|---|---|
| **MySQL Server 8** (InnoDB storage engine) | Relational database management system used for tables, constraints, triggers, stored procedures, functions, views and users. The system has been tested on MySQL 8.0 (Windows) and 26.10.0 (macOS). |
| **MySQL Workbench** | Development and execution of SQL statements, inspection of result sets, generation of the ER diagram by reverse engineering, and demonstration of the project. |
| **Structured Query Language** (MySQL dialect) | Definition of the complete schema and business logic. |
| **Python 3.12, FastAPI and PyMySQL** | Application programming interface between the web pages and the database. All queries are parameterised and no object-relational mapper is employed, so that the SQL remains explicit. |
| **HTML, CSS and JavaScript** | User interfaces for students (booking and rating) and for the management of the messes (dashboard). |
| **Git and GitHub** | Version control and automated verification of the installation procedure on Windows. |
| **pytest** | Automated tests (twenty-three in number) verifying the rules enforced by the database. |

# 5. ER Diagram and Relational Model

## 5.1 Entities and Attributes

| Entity (table) | Description | Principal attributes |
|---|---|---|
| programmes | Academic programme of study | id (PK), code (UNIQUE), name, school, years |
| hostels | Hostel block; B1–B12 for male and G1–G10 for female students | id (PK), name (UNIQUE), for_gender, rooms |
| students | Student enrolled with the university | id (PK), reg_no, phone, email (UNIQUE), programme_id (FK), hostel_id (FK), room_no |
| logins | Successful login of a student | id (PK), student_id (FK), firebase_uid, at |
| messes | Mess (BluDove or BluSpring) | id (PK), name (UNIQUE) |
| meal_slots | Meal of the day with its timing | id (PK), starts_at, ends_at, paid_per_plate |
| mess_costs | Cost of preparing one plate at a mess for a meal | (mess_id, slot_id) composite PK; both FK |
| menu_items | Dish offered by a mess at a meal | id (PK), mess_id (FK), slot_id (FK), dish |
| bookings | Selection made by a student for a meal on a date | id (PK), UNIQUE (student_id, meal_date, slot_id), status |
| pauses | Period of leave of a student | id (PK), student_id (FK), from_date, to_date, ended_on |
| attendance | Meal actually served to a student | id (PK), UNIQUE (student_id, meal_date, slot_id), mess_id (FK) |
| ratings | Rating of a meal by a student who ate it | id (PK), UNIQUE (student_id, meal_date, slot_id), mess_id (FK), stars (CHECK 1–5) |
| ngos | Partner non-governmental organisation | id (PK), name (UNIQUE), area |
| donations | Surplus of a mess for a meal, donated to an NGO | id (PK), UNIQUE (meal_date, slot_id, mess_id), ngo_id (FK) |
| activity_log | Audit record, written exclusively by triggers | id (PK), at, event, student_id, detail |

## 5.2 Relationships

All relationships in the model are of the one-to-many type. A programme and a hostel are each associated with many students. A student is associated with many bookings, periods of leave, attendance records, ratings and logins. A mess and a meal slot each participate in many menu items, bookings, attendance records, ratings and donations, and an NGO receives many donations. The relationship between messes and meal slots is many-to-many with respect to cost; it is resolved by the associative entity `mess_costs`, whose primary key is the composite of the two foreign keys. Each connector in the following diagram corresponds to a FOREIGN KEY constraint defined in the database.

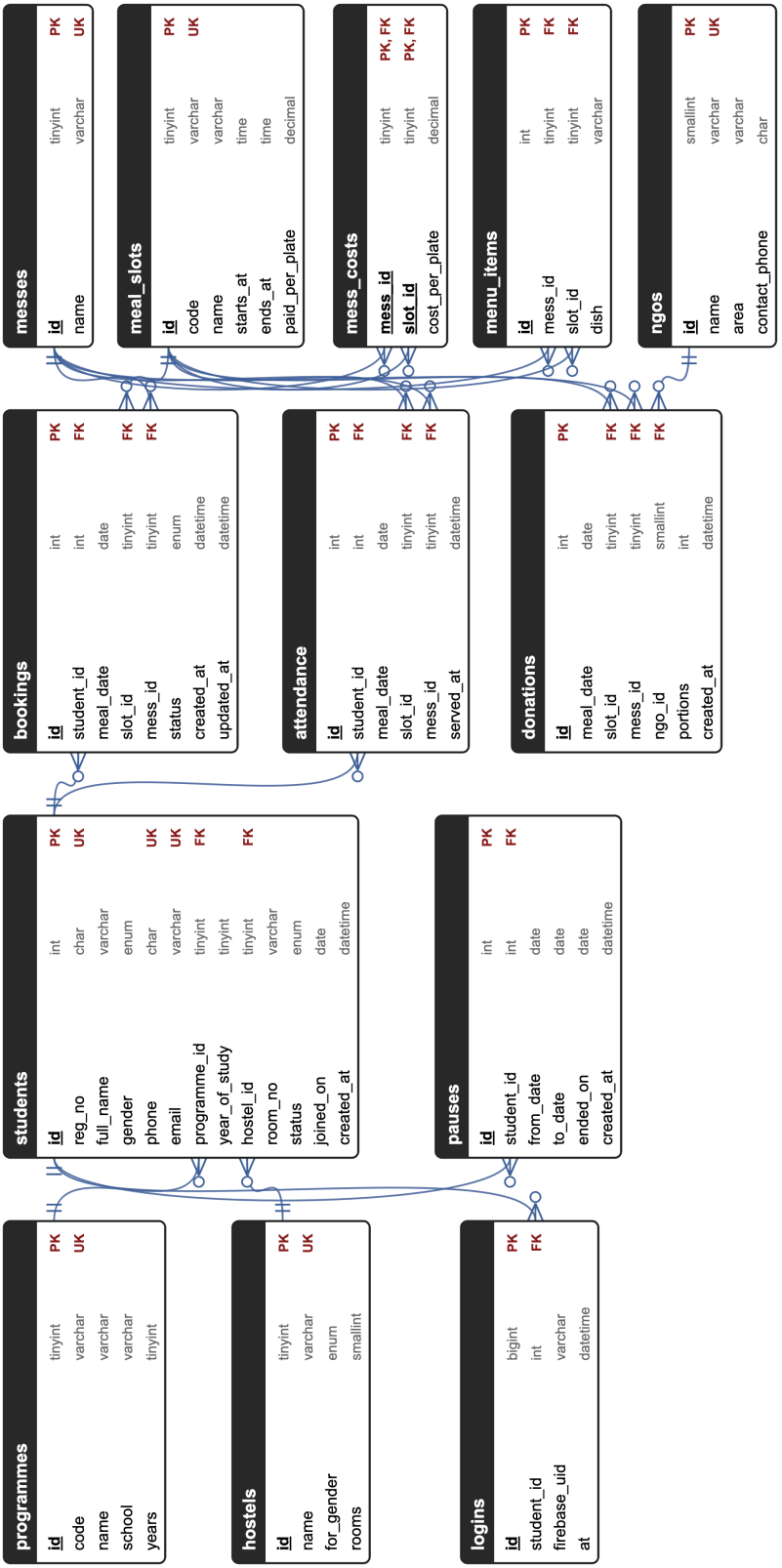



*Figure 1: Entity-Relationship diagram of the database (PK: primary key, FK: foreign key, UK: unique key). The table activity_log is omitted, as it is an audit table without foreign keys.*

## 5.3 Relational Schema

In the following schema, primary keys are underlined and foreign keys are shown in italics and marked with an asterisk (*).

**activity_log**(id, at, event, student_id, detail)

**attendance**(id, *student_id**, meal_date, *slot_id**, *mess_id**, served_at)

**bookings**(id, *student_id**, meal_date, *slot_id**, *mess_id**, status, created_at, updated_at)

**donations**(id, meal_date, *slot_id**, *mess_id**, *ngo_id**, portions, created_at)

**hostels**(id, name, for_gender, rooms)

**logins**(id, *student_id**, firebase_uid, at)

**meal_slots**(id, code, name, starts_at, ends_at, paid_per_plate)

**menu_items**(id, *mess_id**, *slot_id**, dish)

**mess_costs**(*mess_id**, *slot_id**, cost_per_plate)

**messes**(id, name)

**ngos**(id, name, area, contact_phone)

**pauses**(id, *student_id**, from_date, to_date, ended_on, created_at)

**programmes**(id, code, name, school, years)

**ratings**(id, *student_id**, meal_date, *slot_id**, *mess_id**, stars, comment, rated_at)

**students**(id, reg_no, full_name, gender, phone, email, *programme_id**, year_of_study, *hostel_id**, room_no, status, joined_on, created_at)

## 5.4 Foreign Keys

| Foreign key | References |
|---|---|
| attendance.student_id | students.id |
| attendance.slot_id | meal_slots.id |
| attendance.mess_id | messes.id |
| bookings.student_id | students.id |
| bookings.slot_id | meal_slots.id |
| bookings.mess_id | messes.id |
| donations.slot_id | meal_slots.id |
| donations.mess_id | messes.id |
| donations.ngo_id | ngos.id |
| logins.student_id | students.id |
| menu_items.mess_id | messes.id |

| | |
|---|---|
| menu_items.slot_id | meal_slots.id |
| mess_costs.mess_id | messes.id |
| mess_costs.slot_id | meal_slots.id |
| pauses.student_id | students.id |
| ratings.student_id | students.id |
| ratings.slot_id | meal_slots.id |
| ratings.mess_id | messes.id |
| students.programme_id | programmes.id |
| students.hostel_id | hostels.id |

## 5.5 Normalisation

**First Normal Form (1NF):** every attribute holds an atomic value. The dishes of a meal are stored as separate rows of `menu_items` rather than as a list, and the four meals of a student are stored as four rows of `bookings` rather than as four columns. **Second Normal Form (2NF):** no non-key attribute depends on a part of a composite key. In `mess_costs`, whose key is (mess_id, slot_id), the attribute cost_per_plate depends on the complete key. **Third Normal Form (3NF):** no non-key attribute depends transitively on the key. The gender of a hostel is stored only in `hostels` and is not repeated in `students`. Derived quantities, such as the quantity to be prepared, the profit and the last login of a student, are computed through views and are not stored, which eliminates the possibility of inconsistency.

# 6. Database Implementation

## 6.1 Table Creation, Data Types and Constraints

All tables employ the InnoDB storage engine, which supports foreign keys and transactions, and the `utf8mb4` character set. Data types have been selected according to the nature of the data: `TINYINT UNSIGNED` for small reference tables, `INT UNSIGNED` for students and bookings, `BIGINT` for the audit log, `CHAR(10)` for mobile numbers, `DECIMAL(6,2)` for monetary values, `DATE`, `TIME` and `DATETIME(3)` for temporal values, and `ENUM` for attributes with a fixed set of values. The definitions of the two principal tables are reproduced below.

```
CREATE TABLE students (
  id             INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  reg_no         CHAR(9) NOT NULL UNIQUE,
  full_name      VARCHAR(80) NOT NULL,
  gender         ENUM('M','F') NOT NULL,
  phone          CHAR(10) NOT NULL UNIQUE,
  email          VARCHAR(120) NOT NULL UNIQUE,
  programme_id   TINYINT UNSIGNED NOT NULL,
  year_of_study  TINYINT UNSIGNED NOT NULL,
  hostel_id      TINYINT UNSIGNED NOT NULL,
  room_no        VARCHAR(6) NOT NULL,
  status         ENUM('active','graduated','left') NOT NULL DEFAULT 'active',
  joined_on      DATE NOT NULL,
  created_at     DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  INDEX idx_student_hostel (hostel_id),
  INDEX idx_student_programme (programme_id, year_of_study),
  FOREIGN KEY (programme_id) REFERENCES programmes(id),
  FOREIGN KEY (hostel_id)    REFERENCES hostels(id),
  CONSTRAINT chk_phone CHECK (REGEXP_LIKE(phone, '^[6-9][0-9]{9}$')),
  CONSTRAINT chk_reg_no CHECK (REGEXP_LIKE(reg_no, '^[0-9]{9}$')),
  CONSTRAINT chk_room CHECK (REGEXP_LIKE(room_no, '^[0-9]{3,4}$'))
);
```

```
CREATE TABLE bookings (
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  student_id  INT UNSIGNED NOT NULL,
  meal_date   DATE NOT NULL,
  slot_id     TINYINT UNSIGNED NOT NULL,
  mess_id     TINYINT UNSIGNED NULL,
  status      ENUM('booked','skipped','paused') NOT NULL,
  created_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
  UNIQUE KEY uq_one_choice (student_id, meal_date, slot_id),
  INDEX idx_forecast (meal_date, slot_id, mess_id, status),
  FOREIGN KEY (student_id) REFERENCES students(id),
  FOREIGN KEY (slot_id)    REFERENCES meal_slots(id),
  FOREIGN KEY (mess_id)    REFERENCES messes(id),
  CONSTRAINT chk_status_mess CHECK ((status = 'booked') = (mess_id IS NOT NULL))
);
```

| Constraint | Instance in the database | Purpose |
|---|---|---|
| PRIMARY KEY | Every table; composite key in mess_costs | Unique identification of each row |
| FOREIGN KEY | bookings.student_id → students.id (twenty in total) | Referential integrity |
| UNIQUE | bookings (student_id, meal_date, slot_id) | Only one selection per student per meal |
| CHECK | chk_phone, chk_status_mess, chk_pause_range, chk_slot_time, chk_stars | Validity of mobile numbers, consistency of status, valid date ranges and timings, ratings from one to five |
| NOT NULL / DEFAULT | students.hostel_id NOT NULL; created_at DEFAULT CURRENT_TIMESTAMP(3) | Completeness of data |
| Triggers | trg_student_check_ins, trg_booking_paused_ins, trg_rating_check_ins | Rules involving more than one table, which cannot be expressed as CHECK constraints |

## 6.2 Population of the Database

The installation script loads the reference data, namely two messes, four meals, 31 dishes, 12 academic programmes, 22 hostels and three NGOs, together with a roster of 1,330 students. The stored procedure `sp_demo_week` subsequently generates the activity of one week entirely in SQL. At the time of preparation of this report, the database contains 28,315 bookings, 17,039 records of meals served, 4,448 ratings, 54 donations and 56,345 audit records.

## 6.3 Data Manipulation Performed by the Application

```
-- Booking or modification of a meal (one row per student per meal, maintained
by the UNIQUE key)
INSERT INTO bookings (student_id, meal_date, slot_id, mess_id, status)
VALUES (%s, %s, %s, %s, %s) AS new
ON DUPLICATE KEY UPDATE mess_id = new.mess_id, status = new.status;

-- Recording of a meal served (performed by the face-recognition terminal)
INSERT INTO attendance (student_id, meal_date, slot_id, mess_id) VALUES (%s, %s,
%s, %s);

-- Rating of a meal eaten (admitted only if the attendance record exists)
INSERT INTO ratings (student_id, meal_date, slot_id, mess_id, stars, comment)
VALUES (%s, %s, %s, %s, %s, %s) AS new
ON DUPLICATE KEY UPDATE stars = new.stars, comment = new.comment;

-- Recording of a login (invokes the LOGIN trigger)
INSERT INTO logins (student_id, firebase_uid) VALUES (%s, %s);
```

The placeholders (`%s`) are bound by the database driver and are never concatenated into the SQL text; the application is therefore not susceptible to SQL injection.

# 7. SQL Queries and Outputs

Each query presented in this section was executed on the project database, and the result shown is the output obtained.

## 7.1 Data Definition and Database Structure

```
SHOW FULL TABLES
```

| | Tables_in_mess | Table_type |
|---|---|---|
| ▸ | activity_log | BASE TABLE |
| ▸ | attendance | BASE TABLE |
| ▸ | bookings | BASE TABLE |
| ▸ | donations | BASE TABLE |
| ▸ | hostels | BASE TABLE |
| ▸ | logins | BASE TABLE |
| ▸ | meal_slots | BASE TABLE |
| ▸ | menu_items | BASE TABLE |
| ▸ | mess_costs | BASE TABLE |
| ▸ | messes | BASE TABLE |
| ▸ | ngos | BASE TABLE |
| ▸ | pauses | BASE TABLE |
| ▸ | programmes | BASE TABLE |
| ▸ | ratings | BASE TABLE |
| ▸ | students | BASE TABLE |
| ▸ | v_daily_forecast | VIEW |
| ▸ | v_hostel_split | VIEW |
| ▸ | v_live_bookings | VIEW |
| ▸ | v_meal_summary | VIEW |
| ▸ | v_mess_earnings | VIEW |
| ▸ | v_mess_ratings | VIEW |

23 row(s) returned, first 21 shown

*Figure 2: List of tables and views in the database*

```
DESCRIBE students
```

| | Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|---|
| ▸ | id | int unsigned | NO | PRI | *NULL* | auto_increm |
| ▸ | reg_no | char(9) | NO | UNI | *NULL* | |
| ▸ | full_name | varchar(80) | NO | | *NULL* | |
| ▸ | gender | enum('M','F') | NO | | *NULL* | |
| ▸ | phone | char(10) | NO | UNI | *NULL* | |
| ▸ | email | varchar(120) | NO | UNI | *NULL* | |
| ▸ | programme_id | tinyint unsigned | NO | MUL | *NULL* | |
| ▸ | year_of_study | tinyint unsigned | NO | | *NULL* | |
| ▸ | hostel_id | tinyint unsigned | NO | MUL | *NULL* | |
| ▸ | room_no | varchar(6) | NO | | *NULL* | |
| ▸ | status | enum('active','graduated','left') | NO | | active | |
| ▸ | joined_on | date | NO | | *NULL* | |
| ▸ | created_at | datetime(3) | NO | | CURRENT_TIMESTAMP(3) | DEFAULT_C |

13 row(s) returned

*Figure 3: Structure of the table students*

```
ALTER TABLE students ADD COLUMN blood_group VARCHAR(3) NULL AFTER gender;
SELECT column_name, column_type, is_nullable FROM
information_schema.columns
WHERE table_schema = 'mess' AND table_name = 'students' AND column_name =
'blood_group';
ALTER TABLE students DROP COLUMN blood_group
```

✔ ALTER … 0 row(s) affected

| | COLUMN_NAME | COLUMN_TYPE | IS_NULLABLE |
|---|---|---|---|
| ▸ | blood_group | varchar(3) | YES |

1 row(s) returned

✔ ALTER … 0 row(s) affected

*Figure 4: DDL: addition and removal of a column with ALTER TABLE*

```
SELECT (SELECT COUNT(*) FROM students) AS students,
       (SELECT COUNT(*) FROM bookings) AS bookings,
       (SELECT COUNT(*) FROM attendance) AS meals_served,
       (SELECT COUNT(*) FROM donations) AS donations,
       (SELECT COUNT(*) FROM activity_log) AS log_rows
```

| | students | bookings | meals_served | donations | log_rows |
|---|---|---|---|---|---|
| ▸ | 1330 | 28315 | 17039 | 54 | 56345 |

1 row(s) returned

*Figure 5: Number of rows in the principal tables*

## 7.2 Data Manipulation: Insertion, Updation and Deletion

```
INSERT INTO students (reg_no, full_name, gender, phone, email,
programme_id,
                      year_of_study, hostel_id, room_no, joined_on)
VALUES ('250105227', 'Rohan Mehta', 'M', '9876500001',
        'rohan.250105227@muj.manipal.edu', 1, 2, 4, '412', '2025-07-21');
SELECT reg_no, full_name, programme, year_of_study, hostel, room_no
FROM v_student_directory WHERE phone = '9876500001'
```

✔ INSERT … 1 row(s) affected

| | reg_no | full_name | programme | year_of_study | hostel | room_no |
|---|---|---|---|---|---|---|
| ▸ | 250105227 | Rohan Mehta | BTECH-CSE | 2 | B4 | 412 |

1 row(s) returned

*Figure 6: Insertion of a student record and its retrieval through a view*

```
UPDATE students SET room_no = '415' WHERE phone = '9876500001';
SELECT event, detail FROM activity_log
WHERE student_id = (SELECT id FROM students WHERE phone = '9876500001')
```

✔ UPDATE … 1 row(s) affected

| | event | detail |
|---|---|---|
| ▸ | ENROL | Rohan M. · BTECH-CSE year 2 · B4 room 412 |

1 row(s) returned

*Figure 7: Updation of a record and the corresponding audit entry generated by a trigger*

```
INSERT INTO ratings (student_id, meal_date, slot_id, mess_id, stars)
VALUES (1, CURDATE(), 2, 2, 4);
SELECT r.stars, ms.name AS meal, m.name AS mess, a.served_at
FROM ratings r
JOIN attendance a  ON a.student_id = r.student_id AND a.meal_date =
r.meal_date AND a.slot_id = r.slot_id
JOIN meal_slots ms ON ms.id = r.slot_id
JOIN messes m      ON m.id = r.mess_id
WHERE r.student_id = 1 AND r.meal_date = CURDATE()
```

✔ INSERT … 1 row(s) affected

| | stars | meal | mess | served_at |
|---|---|---|---|---|
| ▸ | 4 | Breakfast | BluSpring | 2026-10-27 19:41:48 |
| ▸ | 4 | Lunch | BluSpring | 2026-10-27 19:41:48 |

2 row(s) returned

*Figure 8: Insertion of a rating for a meal that the student has eaten*

```
DELETE FROM ratings
WHERE student_id = 1 AND meal_date = CURDATE() AND slot_id = 2;
SELECT COUNT(*) AS remaining FROM ratings
WHERE student_id = 1 AND meal_date = CURDATE()
```

✔ DELETE … 1 row(s) affected

| | remaining |
|---|---|
| ▸ | 1 |

1 row(s) returned

*Figure 9: DML: deletion of a record with DELETE*

## 7.3 Selection and Joins

```
SELECT reg_no, full_name, year_of_study, room_no
FROM students
WHERE hostel_id = (SELECT id FROM hostels WHERE name = 'G1') AND
year_of_study = 2
ORDER BY full_name
LIMIT 6
```

| | reg_no | full_name | year_of_study | room_no |
|---|---|---|---|---|
| ▸ | 250100464 | Aadhya Meena | 2 | 110 |
| ▸ | 250201150 | Aarohi Khanna | 2 | 501 |
| ▸ | 250700845 | Anika Patel | 2 | 606 |
| ▸ | 250800970 | Avni Kulkarni | 2 | 230 |
| ▸ | 250600936 | Avni Tiwari | 2 | 504 |
| ▸ | 251100295 | Divya Bhatia | 2 | 328 |

6 row(s) returned

*Figure 10: Selection with WHERE, ORDER BY and LIMIT clauses*

```
SELECT b.meal_date, ms.name AS meal, COALESCE(m.name, '-') AS mess,
b.status
FROM bookings b
JOIN students s     ON s.id = b.student_id
JOIN meal_slots ms  ON ms.id = b.slot_id
LEFT JOIN messes m  ON m.id = b.mess_id
WHERE s.phone = '7000000001'
ORDER BY b.meal_date DESC, ms.id
LIMIT 8
```

| | meal_date | meal | mess | status |
|---|---|---|---|---|
| ▸ | 2026-10-28 | Breakfast | - | skipped |
| ▸ | 2026-10-28 | Lunch | - | skipped |
| ▸ | 2026-10-28 | Hi-Tea | - | skipped |
| ▸ | 2026-10-28 | Dinner | - | skipped |
| ▸ | 2026-10-27 | Breakfast | BluSpring | booked |
| ▸ | 2026-10-27 | Lunch | BluDove | booked |
| ▸ | 2026-10-27 | Hi-Tea | - | skipped |
| ▸ | 2026-10-27 | Dinner | BluSpring | booked |

8 row(s) returned

*Figure 11: Inner and left outer joins across four tables: bookings of a student*

## 7.4 Grouping, Aggregation and Subqueries

```
SELECT ms.name AS meal,
       SUM(b.mess_id = 1)          AS bludove,
       SUM(b.mess_id = 2)          AS bluspring,
       SUM(b.status = 'skipped')   AS skipped,
       ROUND(100 * SUM(b.mess_id = 1) / SUM(b.status = 'booked'), 1) AS
bludove_pct
FROM bookings b JOIN meal_slots ms ON ms.id = b.slot_id
WHERE b.meal_date = CURDATE() - INTERVAL 1 DAY
GROUP BY ms.id, ms.name
ORDER BY ms.id
```

| | meal | bludove | bluspring | skipped | bludove_pct |
|---|---|---|---|---|---|
| ▸ | Breakfast | 391 | 336 | 107 | 53.8 |
| ▸ | Lunch | 295 | 447 | 69 | 39.8 |
| ▸ | Hi-Tea | 387 | 205 | 241 | 65.4 |
| ▸ | Dinner | 364 | 401 | 78 | 47.6 |

4 row(s) returned

*Figure 12: Grouping with conditional aggregation: distribution of the previous day's bookings between the messes*

```
SELECT h.name AS hostel, COUNT(*) AS plates_booked
FROM bookings b
JOIN students s ON s.id = b.student_id
JOIN hostels h  ON h.id = s.hostel_id
WHERE b.status = 'booked'
GROUP BY h.id, h.name
HAVING COUNT(*) > (SELECT AVG(n) FROM (SELECT COUNT(*) AS n FROM bookings
b2
                   JOIN students s2 ON s2.id = b2.student_id
                   WHERE b2.status = 'booked' GROUP BY s2.hostel_id) x)
ORDER BY plates_booked DESC
```

| | hostel | plates_booked |
|---|---|---|
| ▸ | G1 | 1771 |
| ▸ | G2 | 1344 |
| ▸ | B11 | 1332 |
| ▸ | B8 | 1315 |
| ▸ | B2 | 1248 |
| ▸ | B5 | 1213 |
| ▸ | B7 | 1176 |
| ▸ | B6 | 1174 |

12 row(s) returned, first 8 shown

*Figure 13: HAVING clause with a nested subquery: hostels with bookings above the average*

```
SELECT p.code AS programme, s.year_of_study AS yr, COUNT(*) AS students
FROM students s JOIN programmes p ON p.id = s.programme_id
WHERE p.code IN ('BTECH-CSE', 'BBA')
GROUP BY p.code, s.year_of_study WITH ROLLUP
```

| | programme | yr | students |
|---|---|---|---|
| ▸ | BBA | 1 | 42 |
| ▸ | BBA | 2 | 49 |
| ▸ | BBA | 3 | 37 |
| ▸ | BBA | *NULL* | 128 |
| ▸ | BTECH-CSE | 1 | 97 |
| ▸ | BTECH-CSE | 2 | 218 |
| ▸ | BTECH-CSE | 3 | 95 |
| ▸ | BTECH-CSE | 4 | 114 |
| ▸ | BTECH-CSE | *NULL* | 524 |
| ▸ | *NULL* | *NULL* | 652 |

10 row(s) returned

*Figure 14: GROUP BY with ROLLUP: subtotals by programme and the grand total*

```
SELECT s.reg_no, fn_short_name(s.full_name) AS student, ms.name AS meal,
m.name AS ate_at
FROM attendance a
JOIN students s    ON s.id = a.student_id
JOIN meal_slots ms ON ms.id = a.slot_id
JOIN messes m      ON m.id = a.mess_id
WHERE NOT EXISTS (SELECT 1 FROM bookings b
                  WHERE b.student_id = a.student_id AND b.meal_date =
a.meal_date
                    AND b.slot_id = a.slot_id AND b.mess_id = a.mess_id AND
b.status = 'booked')
LIMIT 6
```

| | reg_no | student | meal | ate_at |
|---|---|---|---|---|
| ▸ | 240700012 | Anika N. | Breakfast | BluSpring |
| ▸ | 260700016 | Arnav R. | Breakfast | BluSpring |
| ▸ | 250800018 | Ira B. | Breakfast | BluSpring |
| ▸ | 240800027 | Rahul M. | Breakfast | BluSpring |
| ▸ | 260400045 | Khushi C. | Breakfast | BluSpring |
| ▸ | 250100060 | Aisha T. | Breakfast | BluSpring |

6 row(s) returned

*Figure 15: Correlated subquery (NOT EXISTS): students served without a booking at that mess*

## 7.5 Functions

```
SELECT full_name,
       fn_short_name(full_name)               AS public_name,
       UPPER(LEFT(full_name, 1))              AS initial,
       CONCAT('+91 ', LEFT(phone, 5), ' ', RIGHT(phone, 5)) AS
phone_pretty,
       TIMESTAMPDIFF(MONTH, joined_on, CURDATE()) AS months_at_muj,
       DATE_FORMAT(joined_on, '%d %b %Y')     AS joined
FROM students ORDER BY id LIMIT 5
```

| | full_name | public_name | initial | phone_pretty | months_at_muj | joined |
|---|---|---|---|---|---|---|
| ▸ | Anvi Das | Anvi D. | A | +91 70000 00001 | 2 | 30 Jul 2026 |
| ▸ | Ansh Nair | Ansh N. | A | +91 70000 00002 | 2 | 29 Jul 2026 |
| ▸ | Siya Malhotra | Siya M. | S | +91 70000 00003 | 26 | 31 Jul 2024 |
| ▸ | Yash Mehta | Yash M. | Y | +91 70000 00004 | 27 | 24 Jul 2024 |
| ▸ | Aditi Goyal | Aditi G. | A | +91 70000 00005 | 3 | 18 Jul 2026 |

5 row(s) returned

*Figure 16: Built-in string and date functions and the user-defined function fn_short_name*

## 7.6 Views

The database defines seven views: `v_daily_forecast`, `v_hostel_split`, `v_live_bookings`, `v_meal_summary`, `v_mess_earnings`, `v_mess_ratings`, `v_ngo_totals`, `v_student_directory`. All computations are encapsulated in these views, so that the web application and MySQL Workbench obtain identical and current results.

```
SELECT meal, mess, booked, cook_for
FROM v_daily_forecast
WHERE meal_date = CURDATE()
ORDER BY slot_id, mess_id
```

| | meal | mess | booked | cook_for |
|---|---|---|---|---|
| ▸ | Breakfast | BluDove | 602 | 783 |
| ▸ | Breakfast | BluSpring | 442 | 575 |
| ▸ | Lunch | BluDove | 439 | 571 |
| ▸ | Lunch | BluSpring | 677 | 881 |
| ▸ | Hi-Tea | BluDove | 544 | 708 |
| ▸ | Hi-Tea | BluSpring | 324 | 422 |
| ▸ | Dinner | BluDove | 518 | 674 |
| ▸ | Dinner | BluSpring | 574 | 747 |

8 row(s) returned

*Figure 17: View v_daily_forecast: quantity to be prepared by each mess for the current day*

```
SELECT meal, mess, booked, cooked, served, no_shows, walk_ins, donate
FROM v_meal_summary
WHERE meal_date = CURDATE() - INTERVAL 1 DAY
ORDER BY slot_id, mess_id
```

| | meal | mess | booked | cooked | served | no_shows | walk_ins | donate |
|---|---|---|---|---|---|---|---|---|
| ▸ | Breakfast | BluDove | 391 | 509 | 336 | 55 | 0 | 145 |
| ▸ | Breakfast | BluSpring | 336 | 437 | 266 | 70 | 0 | 136 |
| ▸ | Lunch | BluDove | 295 | 384 | 251 | 44 | 0 | 111 |
| ▸ | Lunch | BluSpring | 447 | 582 | 381 | 66 | 0 | 168 |
| ▸ | Hi-Tea | BluDove | 387 | 504 | 324 | 63 | 0 | 148 |
| ▸ | Hi-Tea | BluSpring | 205 | 267 | 173 | 32 | 0 | 78 |
| ▸ | Dinner | BluDove | 364 | 474 | 316 | 48 | 0 | 134 |
| ▸ | Dinner | BluSpring | 401 | 522 | 340 | 61 | 0 | 151 |

8 row(s) returned

*Figure 18: View v_meal_summary: meals served, absentees, walk-ins and surplus for donation*

```
SELECT mess,
       SUM(served)    AS plates_served,
       SUM(revenue)   AS revenue,
       SUM(food_cost) AS food_cost,
       SUM(profit)    AS profit,
       ROUND(100 * SUM(profit) / SUM(revenue), 1) AS margin_pct
FROM v_mess_earnings
WHERE meal_date BETWEEN CURDATE() - INTERVAL 6 DAY AND CURDATE() - INTERVAL
1 DAY
GROUP BY mess_id, mess
```

| | mess | plates_served | revenue | food_cost | profit | margin_pct |
|---|---|---|---|---|---|---|
| ▸ | BluDove | 7,275.00 | 504,795.00 | 434,650.00 | 70,145.00 | 13.9 |
| ▸ | BluSpring | 7,305.00 | 548,705.00 | 467,134.00 | 81,571.00 | 14.9 |

2 row(s) returned

*Figure 19: View v_mess_earnings: revenue, cost and profit of each mess over six days*

```
SELECT meal, mess,
       ROUND(SUM(avg_stars * ratings) / SUM(ratings), 2) AS avg_stars,
       SUM(ratings) AS ratings, SUM(unhappy) AS one_or_two_stars
FROM v_mess_ratings
WHERE meal_date >= CURDATE() - INTERVAL 6 DAY
GROUP BY slot_id, meal, mess_id, mess
ORDER BY slot_id, mess_id
```

| | meal | mess | avg_stars | ratings | one_or_two_stars |
|---|---|---|---|---|---|
| ▸ | Breakfast | BluDove | 4.08 | 619 | 2 |
| ▸ | Breakfast | BluSpring | 3.66 | 518 | 22 |
| ▸ | Lunch | BluDove | 3.63 | 499 | 10 |
| ▸ | Lunch | BluSpring | 4.31 | 742 | 1 |
| ▸ | Hi-Tea | BluDove | 3.40 | 521 | 65 |
| ▸ | Hi-Tea | BluSpring | 3.96 | 325 | 0 |
| ▸ | Dinner | BluDove | 4.29 | 561 | 0 |
| ▸ | Dinner | BluSpring | 3.51 | 663 | 54 |

8 row(s) returned

*Figure 20: View v_mess_ratings: average rating of each mess for each meal over seven days*

```
SELECT hostel, mess, SUM(booked) AS plates
FROM v_hostel_split
WHERE meal_date = CURDATE() AND hostel IN ('B1', 'B2', 'G1', 'G2')
GROUP BY hostel_id, hostel, mess
ORDER BY hostel_id, mess
```

| | hostel | mess | plates |
|---|---|---|---|
| ▸ | B1 | BluDove | 101 |
| ▸ | B1 | BluSpring | 99 |
| ▸ | B2 | BluDove | 99 |
| ▸ | B2 | BluSpring | 103 |
| ▸ | G1 | BluDove | 252 |
| ▸ | G1 | BluSpring | 225 |
| ▸ | G2 | BluDove | 101 |
| ▸ | G2 | BluSpring | 113 |

8 row(s) returned

*Figure 21: View v_hostel_split: distribution of hostels between the two messes*

# 8. Advanced Database Operations

## 8.1 Transactions

A transaction groups a set of statements into a single atomic unit of work. In the example below, the costs of BluDove are increased within a transaction, which is then rolled back; the database is consequently restored to its previous state. Every stored procedure of the project begins with `START TRANSACTION` and declares an `EXIT HANDLER` that executes `ROLLBACK` in the event of an error.

```
START TRANSACTION;
UPDATE mess_costs SET cost_per_plate = cost_per_plate * 1.10 WHERE mess_id
= 1;
SELECT slot_id, cost_per_plate FROM mess_costs WHERE mess_id = 1;
ROLLBACK;
SELECT slot_id, cost_per_plate FROM mess_costs WHERE mess_id = 1
```

✔ START … 0 row(s) affected

✔ UPDATE … 4 row(s) affected

| | slot_id | cost_per_plate |
|---|---|---|
| ▸ | 1 | 34.10 |
| ▸ | 2 | 53.90 |
| ▸ | 3 | 19.80 |
| ▸ | 4 | 66.00 |

4 row(s) returned

✔ ROLLBACK … 0 row(s) affected

| | slot_id | cost_per_plate |
|---|---|---|
| ▸ | 1 | 31.00 |
| ▸ | 2 | 49.00 |
| ▸ | 3 | 18.00 |
| ▸ | 4 | 60.00 |

4 row(s) returned

*Figure 22: Transaction control: modification followed by ROLLBACK*

A savepoint marks a point within a transaction to which the transaction may be partially rolled back:

```
START TRANSACTION;
UPDATE meal_slots SET paid_per_plate = paid_per_plate + 5 WHERE id = 1;
SAVEPOINT after_breakfast;
UPDATE meal_slots SET paid_per_plate = paid_per_plate + 5 WHERE id = 2;
ROLLBACK TO SAVEPOINT after_breakfast;
SELECT id, name, paid_per_plate FROM meal_slots WHERE id IN (1, 2);
ROLLBACK
```

✔ START … 0 row(s) affected

✔ UPDATE … 1 row(s) affected

✔ SAVEPOINT … 0 row(s) affected

✔ UPDATE … 1 row(s) affected

✔ ROLLBACK … 0 row(s) affected

| | id | name | paid_per_plate |
|---|---|---|---|
| ▸ | 1 | Breakfast | 65.00 |
| ▸ | 2 | Lunch | 85.00 |

2 row(s) returned

✔ ROLLBACK … 0 row(s) affected

*Figure 23: Partial rollback with SAVEPOINT: only the second update is undone*

## 8.2 Triggers

A trigger is executed automatically when a row of its table is modified. Triggers serve two purposes in this project. BEFORE triggers enforce business rules and reject invalid data by raising `SIGNAL SQLSTATE '45000'`. AFTER triggers maintain an audit trail by recording every enrolment, login, booking, modification, leave, meal served, rating and donation in the table `activity_log`.

```
CREATE TRIGGER trg_booking_paused_ins BEFORE INSERT ON bookings FOR EACH ROW
BEGIN
  IF NEW.status = 'booked' AND EXISTS (
       SELECT 1 FROM pauses p
       WHERE p.student_id = NEW.student_id
         AND NEW.meal_date BETWEEN p.from_date AND p.to_date
         AND (p.ended_on IS NULL OR NEW.meal_date < p.ended_on)) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Your mess is paused for this
date';
  END IF;
END$$
```

```
SELECT trigger_name, action_timing AS timing, event_manipulation AS event,
event_object_table AS on_table
FROM information_schema.triggers
WHERE trigger_schema = 'mess'
ORDER BY event_object_table, trigger_name
```

| | TRIGGER_NAME | timing | event | on_table |
|---|---|---|---|---|
| ▸ | trg_attendance_log | AFTER | INSERT | attendance |
| ▸ | trg_booking_log_ins | AFTER | INSERT | bookings |
| ▸ | trg_booking_log_upd | AFTER | UPDATE | bookings |
| ▸ | trg_booking_paused_ins | BEFORE | INSERT | bookings |
| ▸ | trg_booking_paused_upd | BEFORE | UPDATE | bookings |
| ▸ | trg_donation_log | AFTER | INSERT | donations |
| ▸ | trg_login_log | AFTER | INSERT | logins |
| ▸ | trg_pause_log | AFTER | INSERT | pauses |
| ▸ | trg_resume_log | AFTER | UPDATE | pauses |
| ▸ | trg_rating_check_ins | BEFORE | INSERT | ratings |
| ▸ | trg_rating_check_upd | BEFORE | UPDATE | ratings |
| ▸ | trg_rating_log | AFTER | INSERT | ratings |
| ▸ | trg_student_check_ins | BEFORE | INSERT | students |
| ▸ | trg_student_check_upd | BEFORE | UPDATE | students |
| ▸ | trg_student_enrol | AFTER | INSERT | students |

15 row(s) returned

*Figure 24: Triggers defined in the database*

```
INSERT INTO students (reg_no, full_name, gender, phone, email,
programme_id,
                      year_of_study, hostel_id, room_no, joined_on)
VALUES ('230100998', 'Wrong Hostel', 'M', '9876501235',
        'w@muj.manipal.edu', 1, 1, 13, '101', '2023-07-24')
```

✖ Error Code: 1644. This hostel is for the other gender

*Figure 25: Rejection by a BEFORE INSERT trigger: allotment of a female hostel to a male student*

```
INSERT INTO students (reg_no, full_name, gender, phone, email,
programme_id,
                      year_of_study, hostel_id, room_no, joined_on)
VALUES ('230100996', 'Bad Phone', 'M', '12345',
        'p@muj.manipal.edu', 1, 1, 1, '101', '2023-07-24')
```

✖ Error Code: 3819. Check constraint 'chk_phone' is violated.

*Figure 26: Rejection by the CHECK constraint chk_phone: invalid mobile number*

```
INSERT INTO ratings (student_id, meal_date, slot_id, mess_id, stars)
VALUES (1, CURDATE(), 4, 1, 5)
```

✖ Error Code: 1644. You can only rate a meal you have eaten

*Figure 27: Rejection by the trigger trg_rating_check_ins: rating of a meal not eaten*

## 8.3 Stored Procedures and Function

```
CREATE PROCEDURE sp_pause_mess(IN p_student INT UNSIGNED, IN p_from DATE, IN
p_to DATE)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
    INSERT INTO pauses (student_id, from_date, to_date) VALUES (p_student,
p_from, p_to);
    -- Meals already chosen in that range become 'paused' (not counted).
    UPDATE bookings SET status = 'paused', mess_id = NULL
    WHERE student_id = p_student AND meal_date BETWEEN p_from AND p_to;
  COMMIT;
END$$
```

```
SELECT routine_name, routine_type, is_deterministic
FROM information_schema.routines WHERE routine_schema = 'mess'
ORDER BY routine_type, routine_name
```

| | ROUTINE_NAME | ROUTINE_TYPE | IS_DETERMINISTIC |
|---|---|---|---|
| ▸ | fn_short_name | FUNCTION | YES |
| ▸ | sp_check_student | PROCEDURE | NO |
| ▸ | sp_close_finished_meals | PROCEDURE | NO |
| ▸ | sp_close_meal | PROCEDURE | NO |
| ▸ | sp_demo_day | PROCEDURE | NO |
| ▸ | sp_demo_week | PROCEDURE | NO |
| ▸ | sp_pause_mess | PROCEDURE | NO |
| ▸ | sp_resume_mess | PROCEDURE | NO |

8 row(s) returned

*Figure 28: Stored procedures and function defined in the database*

```
SELECT meal_date, slot_id, mess_id, status FROM bookings
WHERE student_id = 21 AND meal_date = CURDATE() ORDER BY slot_id
```

| | meal_date | slot_id | mess_id | status |
|---|---|---|---|---|
| ▸ | 2026-10-27 | 1 | 2 | booked |
| ▸ | 2026-10-27 | 2 | *NULL* | skipped |
| ▸ | 2026-10-27 | 3 | 1 | booked |
| ▸ | 2026-10-27 | 4 | 2 | booked |

4 row(s) returned

*Figure 29: Bookings of a student before the declaration of leave*

```
CALL sp_pause_mess(21, CURDATE(), CURDATE() + INTERVAL 1 DAY);
SELECT meal_date, slot_id, mess_id, status FROM bookings
WHERE student_id = 21 AND meal_date = CURDATE() ORDER BY slot_id
```

✔ CALL … 0 row(s) affected

| | meal_date | slot_id | mess_id | status |
|---|---|---|---|---|
| ▸ | 2026-10-27 | 1 | *NULL* | paused |
| ▸ | 2026-10-27 | 2 | *NULL* | paused |
| ▸ | 2026-10-27 | 3 | *NULL* | paused |
| ▸ | 2026-10-27 | 4 | *NULL* | paused |

4 row(s) returned

*Figure 30: Execution of the stored procedure sp_pause_mess*

```
INSERT INTO bookings (student_id, meal_date, slot_id, mess_id, status)
VALUES (21, CURDATE(), 4, 1, 'booked')
ON DUPLICATE KEY UPDATE mess_id = 1, status = 'booked'
```

✖ Error Code: 1644. Your mess is paused for this date

*Figure 31: Rejection of a booking during leave by a trigger (SIGNAL SQLSTATE '45000')*

```
CALL sp_resume_mess(21, CURDATE());
SELECT event, detail FROM activity_log WHERE student_id = 21 ORDER BY id
DESC LIMIT 4
```

✔ CALL … 0 row(s) affected

| | event | detail |
|---|---|---|
| ▸ | CHANGE | Zoya K. · Dinner · 2026-10-28 · paused → skipped |
| ▸ | CHANGE | Zoya K. · Hi-Tea · 2026-10-28 · paused → skipped |
| ▸ | CHANGE | Zoya K. · Lunch · 2026-10-28 · paused → skipped |
| ▸ | CHANGE | Zoya K. · Breakfast · 2026-10-28 · paused → skipped |

4 row(s) returned

*Figure 32: Execution of sp_resume_mess and the resulting audit entries*

The procedure `sp_close_meal` employs a cursor to iterate over the surplus of each mess for a meal and assigns it, within a single transaction, to the NGO that has received the least quantity so far:

```
CALL sp_close_meal(CURDATE() - INTERVAL 1 DAY, 2);
SELECT n.name AS ngo, n.area, d.portions, m.name AS mess
FROM donations d JOIN ngos n ON n.id = d.ngo_id JOIN messes m ON m.id =
d.mess_id
WHERE d.meal_date = CURDATE() - INTERVAL 1 DAY AND d.slot_id = 2
```

✔ CALL … 0 row(s) affected

| | ngo | area | portions | mess |
|---|---|---|---|---|
| ▸ | Ann Seva Night Shelter | Sanganer | 111 | BluDove |
| ▸ | Dahmi Community Kitchen | Dahmi Kalan | 168 | BluSpring |

2 row(s) returned

*Figure 33: Execution of the stored procedure sp_close_meal: allocation of surplus to NGOs*

## 8.4 User Privileges

The application does not connect to the database as the root user. The installation script creates the user `mess_app` with only those privileges that the application requires; consequently, the application is unable to delete records or to alter or drop any object. In addition, a read-only user is created for the management of the messes, with access restricted to selected views; the REVOKE statement withdraws a privilege previously granted.

```
SHOW GRANTS FOR 'mess_app'@'localhost'
```

| | Grants for mess_app@localhost |
|---|---|
| ▸ | GRANT USAGE ON *.* TO `mess_app`@`localhost` |
| ▸ | GRANT SELECT, INSERT, UPDATE, EXECUTE ON `mess`.* TO `… |

2 row(s) returned

*Figure 34: Privileges granted to the application user mess_app*

```
DROP USER IF EXISTS 'mess_viewer'@'localhost';
CREATE USER 'mess_viewer'@'localhost' IDENTIFIED BY 'Viewer#2026';
GRANT SELECT ON mess.v_daily_forecast TO 'mess_viewer'@'localhost';
GRANT SELECT ON mess.v_mess_earnings TO 'mess_viewer'@'localhost';
GRANT SELECT ON mess.v_mess_ratings TO 'mess_viewer'@'localhost';
REVOKE SELECT ON mess.v_mess_earnings FROM 'mess_viewer'@'localhost';
SHOW GRANTS FOR 'mess_viewer'@'localhost'
```

✔ DROP … 0 row(s) affected

✔ CREATE … 0 row(s) affected

✔ GRANT … 0 row(s) affected

✔ GRANT … 0 row(s) affected

✔ GRANT … 0 row(s) affected

✔ REVOKE … 0 row(s) affected

| | Grants for mess_viewer@localhost |
|---|---|
| ▸ | GRANT USAGE ON *.* TO `mess_viewer`@`localhost` |
| ▸ | GRANT SELECT ON `mess`.`v_daily_forecast` TO `mess_viewer`… |
| ▸ | GRANT SELECT ON `mess`.`v_mess_ratings` TO `mess_viewer`… |

3 row(s) returned

*Figure 35: Creation of a read-only user, GRANT of privileges on views and REVOKE of one privilege*

## 8.5 Scheduled Events

An event is a task executed by the MySQL event scheduler at defined intervals. The event `ev_close_meals` runs every ten minutes and invokes `sp_close_finished_meals`, which uses a cursor to select every meal of the current day whose serving period has ended and that has not yet been closed, and calls `sp_close_meal` for each. The surplus food is thereby allocated to NGOs without any manual intervention.

```
CREATE EVENT ev_close_meals ON SCHEDULE EVERY 10 MINUTE
COMMENT 'Closes finished meals and hands leftovers to NGOs'
DO CALL sp_close_finished_meals()$$
```

```
SELECT event_name, interval_value, interval_field, status, event_definition
FROM information_schema.events WHERE event_schema = 'mess';
CALL sp_close_finished_meals();
SELECT ms.name AS meal, m.name AS mess, n.name AS ngo, d.portions
FROM donations d JOIN meal_slots ms ON ms.id = d.slot_id
JOIN messes m ON m.id = d.mess_id JOIN ngos n ON n.id = d.ngo_id
WHERE d.meal_date = CURDATE() ORDER BY d.slot_id, d.mess_id
```

| | EVENT_NAME | INTERVAL_VALUE | INTERVAL_FIELD | STATUS | EVENT_DEFINITION |
|---|---|---|---|---|---|
| ▸ | ev_close_meals | 10 | MINUTE | ENABLED | CALL sp_close_finished_meals( |

1 row(s) returned

✔ CALL … 0 row(s) affected

| | meal | mess | ngo | portions |
|---|---|---|---|---|
| ▸ | Breakfast | BluDove | Ann Seva Night Shelter | 101 |
| ▸ | Breakfast | BluSpring | Dahmi Community Kitchen | 63 |
| ▸ | Lunch | BluDove | Dahmi Community Kitchen | 97 |
| ▸ | Lunch | BluSpring | Dahmi Community Kitchen | 102 |
| ▸ | Hi-Tea | BluDove | Sahyog Food Shelf | 413 |
| ▸ | Hi-Tea | BluSpring | Ann Seva Night Shelter | 241 |

6 row(s) returned

*Figure 36: Scheduled event ev_close_meals and the procedure it executes*

## 9. Screenshots

The following screenshots depict the system in operation. The student interface and the management dashboard operate on the MySQL database described in the preceding sections.

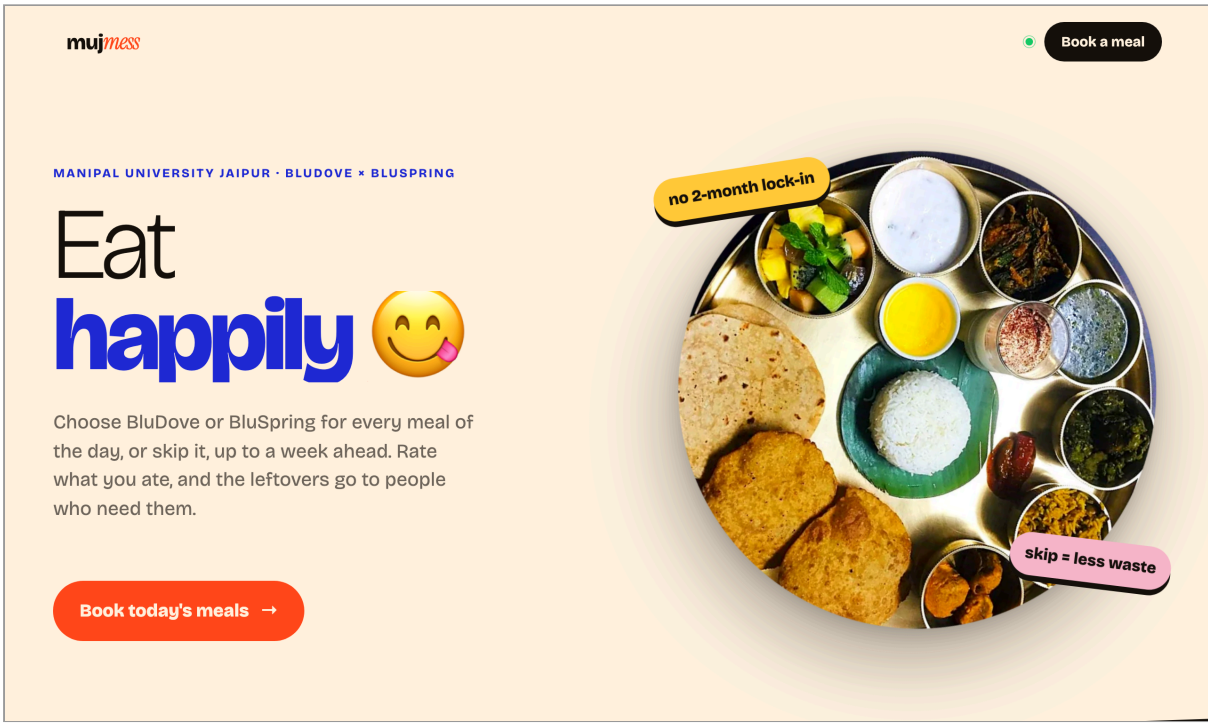



*Figure 37: Home page of the student interface*

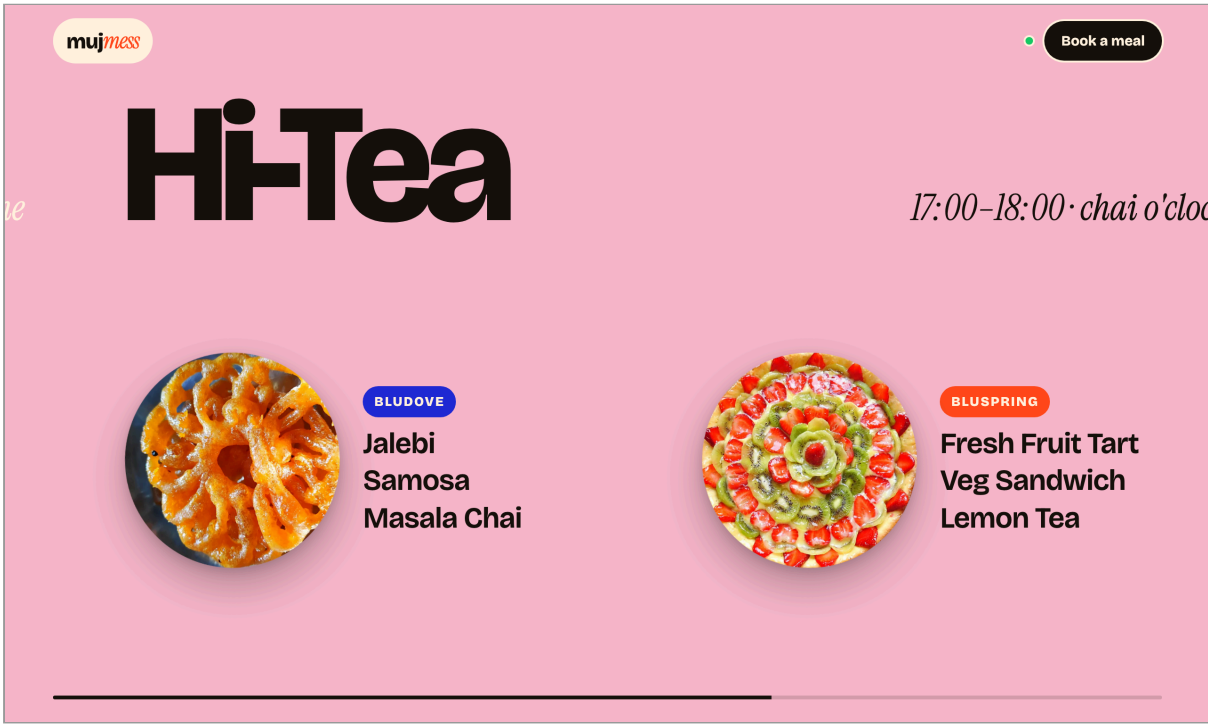



*Figure 38: Menu of both messes for a meal (hi-tea)*

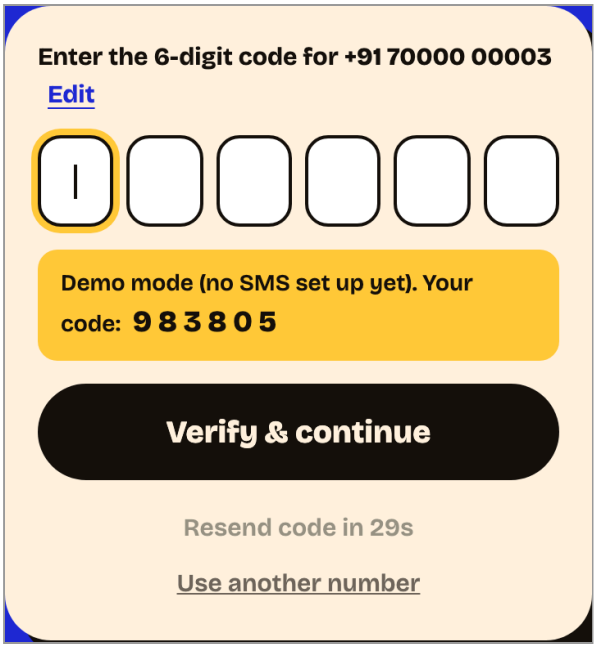



*Figure 39: Login by one-time password*

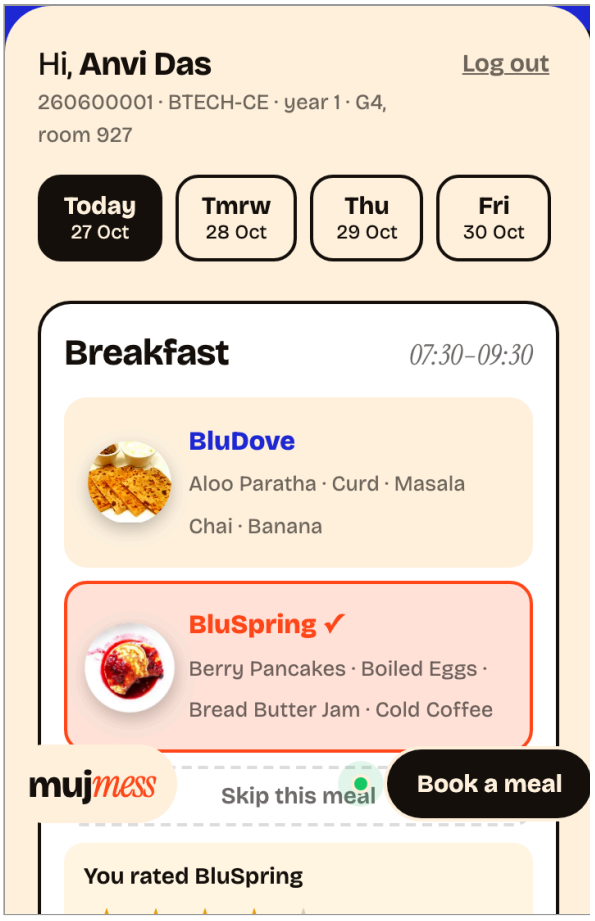



*Figure 40: Meal booking screen with the student profile*

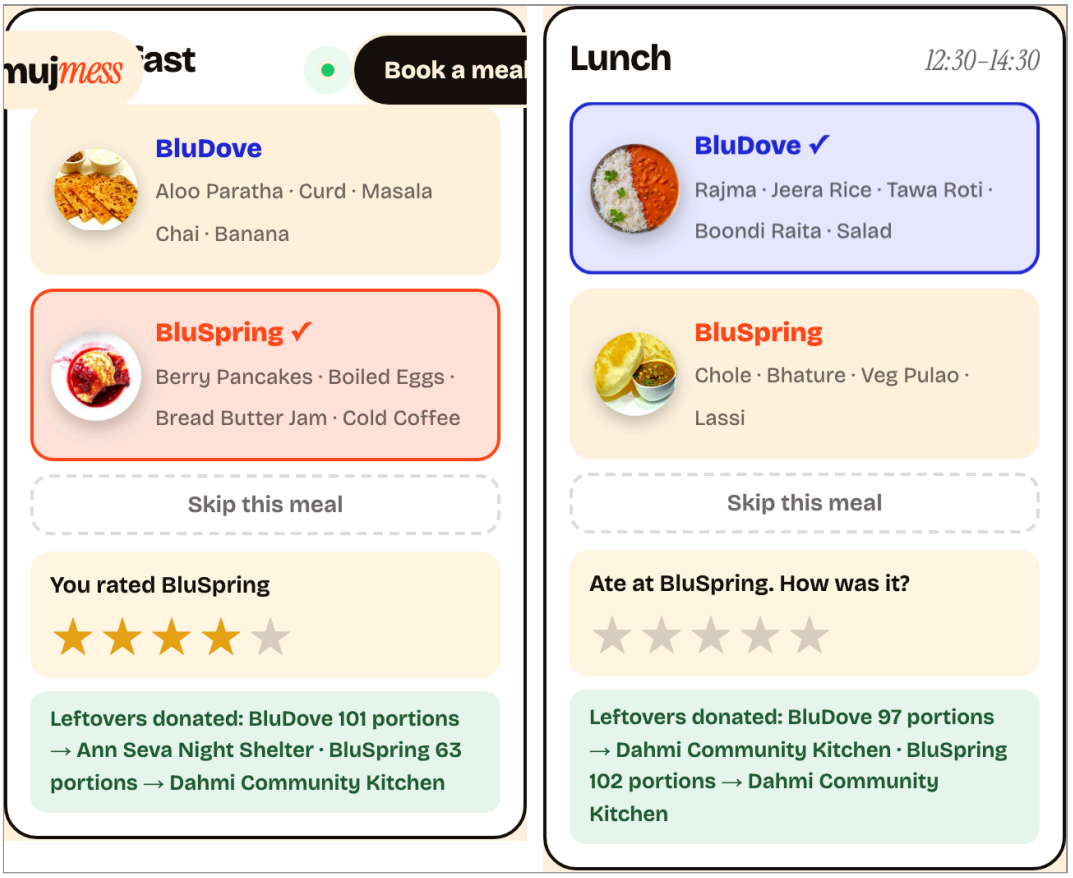
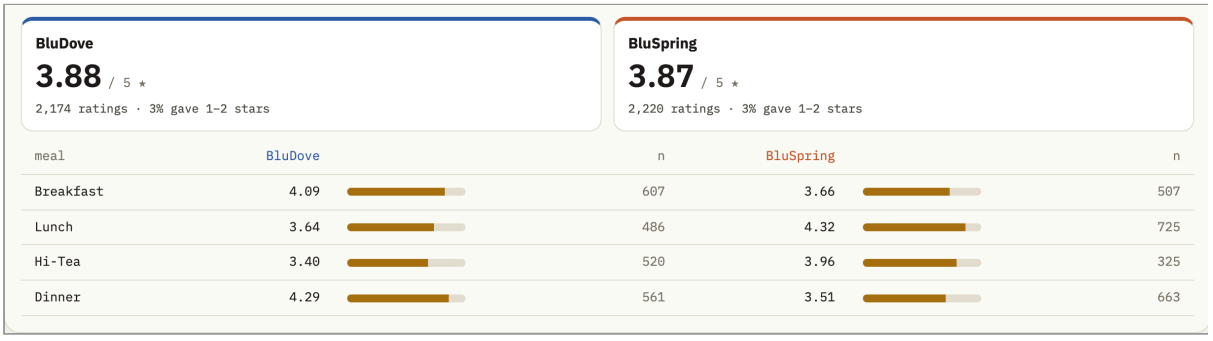



*Figure 41: Rating of meals eaten (a rated breakfast, an unrated lunch) and the donation of leftovers recorded by the scheduled event*




| meal | BluDove | n | BluSpring | n |
|---|---|---|---|---|
| Breakfast | 4.09 | 607 | 3.66 | 507 |
| Lunch | 3.64 | 486 | 4.32 | 725 |
| Hi-Tea | 3.40 | 520 | 3.96 | 325 |
| Dinner | 4.29 | 561 | 3.51 | 663 |

*Figure 42: Ratings board of the management dashboard, obtained from v_mess_ratings*

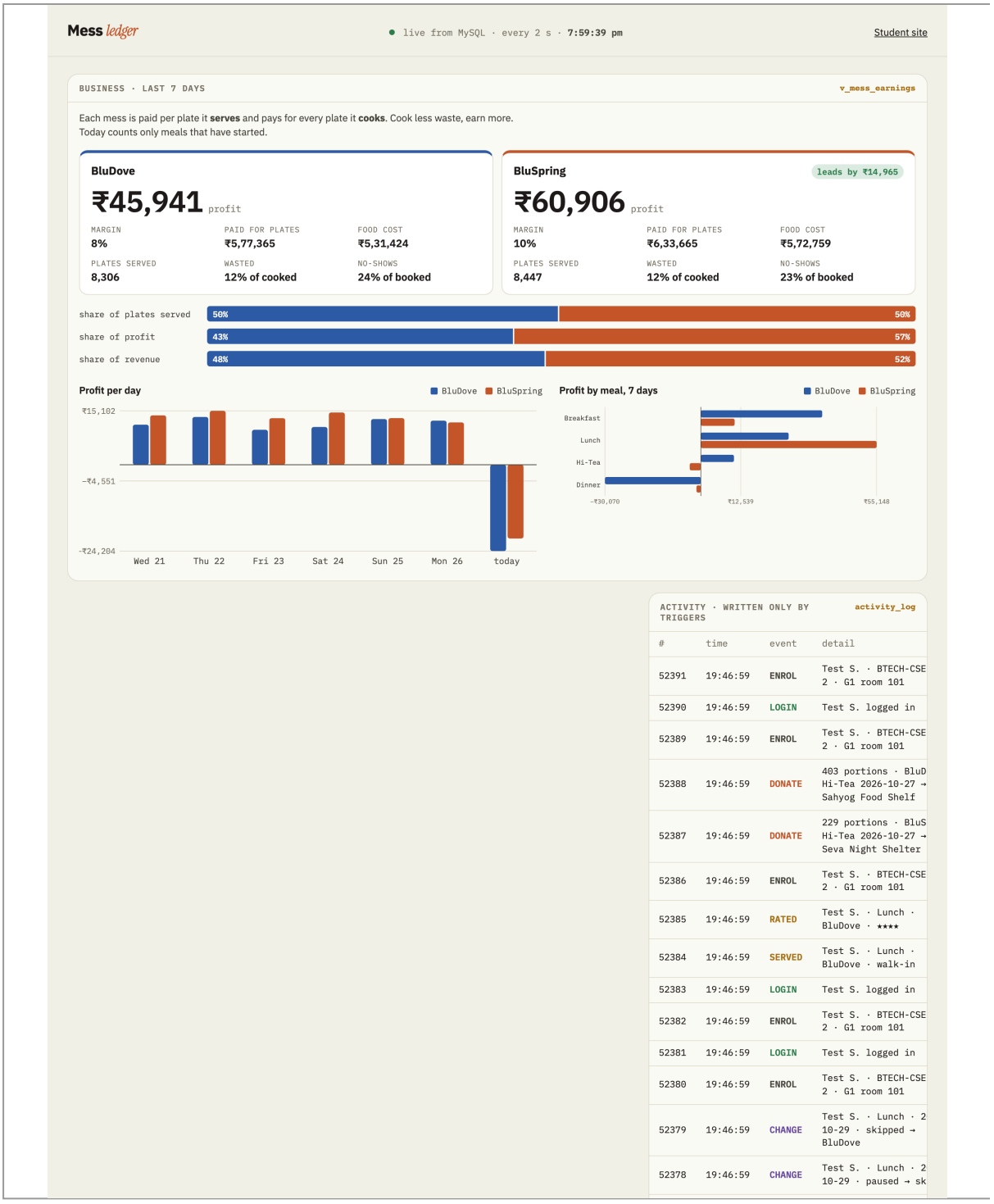



*Figure 43: Management dashboard: profit of each mess (v_mess_earnings), forecast for the following day (v_daily_forecast) and the activity log*

```
SELECT id, DATE_FORMAT(at, '%H:%i:%s') AS time, event, detail
FROM activity_log ORDER BY id DESC LIMIT 8
```

| | id | time | event | detail |
|---|---|---|---|---|
| ▸ | 56367 | 20:25:23 | DONATE | 111 portions · BluDove Lunch 2026-10-26 → Ann Seva Night Shelter |
| ▸ | 56366 | 20:25:23 | DONATE | 168 portions · BluSpring Lunch 2026-10-26 → Dahmi Community … |
| ▸ | 56365 | 20:25:23 | CHANGE | Zoya K. · Dinner · 2026-10-28 · paused → skipped |
| ▸ | 56364 | 20:25:23 | CHANGE | Zoya K. · Hi-Tea · 2026-10-28 · paused → skipped |
| ▸ | 56363 | 20:25:23 | CHANGE | Zoya K. · Lunch · 2026-10-28 · paused → skipped |
| ▸ | 56362 | 20:25:23 | CHANGE | Zoya K. · Breakfast · 2026-10-28 · paused → skipped |
| ▸ | 56361 | 20:25:23 | CHANGE | Zoya K. · Dinner · 2026-10-27 · paused → skipped |
| ▸ | 56360 | 20:25:23 | CHANGE | Zoya K. · Hi-Tea · 2026-10-27 · paused → skipped |

8 row(s) returned

*Figure 44: Audit log maintained by triggers*

# 10. Conclusion

The project has replaced a fixed two-month mess registration with a per-meal choice and has substituted estimation in the kitchens with an exact forecast derived from bookings. The implementation demonstrates that a well-designed relational database can support the entire application: the web interface and the application programming interface constitute thin layers, while the rules, the computations and the history of all operations are maintained within MySQL.

Several lessons were learnt in the course of the implementation. Normalisation to the Third Normal Form eliminated the possibility of inconsistent data, and the use of views for derived quantities ensured correct results without additional program code. Constraints and triggers proved to be the most reliable means of enforcing business rules, since no client application can bypass them. Stored procedures executed within transactions ensured that multi-step operations remain consistent even in the event of failure. The principle of least privilege and the use of parameterised queries made the system secure by design. The project further afforded practical experience with features such as `ON DUPLICATE KEY UPDATE`, correlated subqueries, `WITH ROLLUP` and cursors.

The system may be extended in future by introducing a date-dependent weekly menu, a direct interface for the face-recognition terminals, and menu planning informed by the ratings collected from students.

# 11. References


1. Oracle Corporation, *MySQL 8.0 Reference Manual*. Available: https://dev.mysql.com/doc/refman/8.0/en/
2. Oracle Corporation, *MySQL Workbench Manual*. Available: https://dev.mysql.com/doc/workbench/en/
3. A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York: McGraw-Hill, 2019.
4. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston: Pearson, 2016.
5. C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston: Addison-Wesley, 2003.
6. FastAPI Documentation. Available: https://fastapi.tiangolo.com/
7. PyMySQL Documentation. Available: https://pymysql.readthedocs.io/